# Compiladores

Departamento de Computação
Universidade Federal de Sergipe

# **Análise Léxica**

Prof. Beatriz Trinchão Andrade
beatriz@ufs.br

# Introdução

- Lê um programa fonte e o classifica em *tokens* (marcas)
- Tokens
  - Similares a uma palavra em linguagem natural
  - Exemplos: palavras-chave, identificadores, símbolos especiais

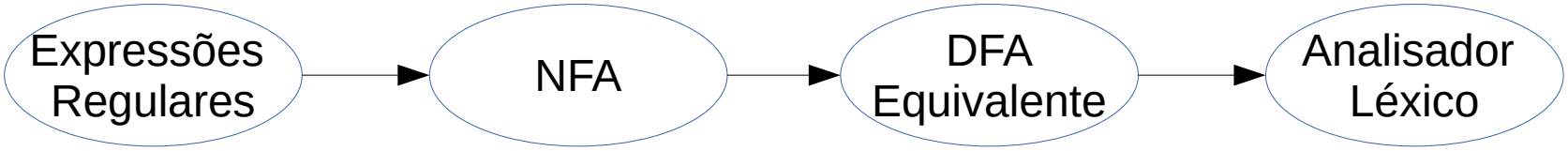
# O Processo de Varredura

- Lexema ou valor
  - Sequência de caracteres
- Token (ou marca)
  - Saída do analisador léxico, resulta da classificação do lexema
- Tokens podem representar um ou mais lexemas

| **Token** | **Lexema** |
|---|---|
| <ID> | x, taxa, i, total, temp |
| <INT> | 235, 0, 10 |
| <REAL> | 235E-23 |
| <IF> | if |
| <COMMA> | , |
| <ASSIGN> | := |
| <LPAR> | ( |
| ... | |

# Construindo um Analisador Léxico

- Construir um DFA (ou NFA) para cada **token** e para os **delimitadores**
- Combinar em um único NFA
- Transformar para DFA
- Implementar o autômato combinado
  - Reconhecer o maior lexema possível



# Expressões Regulares

- Representam padrões de cadeias de caracteres
- Uma expressão regular **r** é completamente definida pelo conjunto de cadeias de caracteres com as quais ela casa
  - Conjunto L(**r**): linguagem (conjunto de cadeias) gerada pela expressão regular
- Alfabeto Σ (sigma): conjunto de símbolos aceitos por expressões regulares básicas
  - Φ → Conjunto vazio
  - ε (epsilon) → Cadeia vazia

# Expressões Regulares

- Expressões regulares sobre Σ
  - Uma E.R. básica que gera uma linguagem composta por um único caractere *a*
    - L(**a**) = {*a*}
  - L(**ε**) = {ε} (cadeia vazia)
  - L(**Φ**) = {} (não casa com nenhuma cadeia)

# Expressões Regulares

- Escolha entre alternativas (união de linguagens):

  **r | s** → L(r | s) = L(r) U L(s)

  Seja **a|ε,** L( **a | ε** ) = ?

- Concatenação: **rs** → L(rs) = L(r)L(s)

  L( **(a | b) c** ) = ?

# Expressões Regulares

- Fecho: **r*** → L(r*) = L(r)*
  - Qualquer número de concatenações finitas de **r**
  - União infinita de conjuntos
  - Inclui **ε**: concatenação de zero cadeias

  L( **(a | bb)***) = ?

# Expressões regulares

- **a|b*** ?
- Expressões regulares sobre Σ
  - Uma expressão na forma **(r)** → L((r)) = L(r)
  - Parênteses não modificam a linguagem, servem apenas para ajustar a precedência dos operadores

# Expressões regulares

- **a|b*** ?
- Expressões regulares sobre Σ
  - Uma expressão na forma **(r)** → L((r)) = L(r)
  - Parênteses não modificam a linguagem, servem apenas para ajustar a precedência dos operadores
- Precedência

  * > concatenação > |

# Expressões Regulares

- Exemplo 1
  - $\Sigma = \{a, b, c\}$
  - Cadeias que contenham exatamente um b
- Exemplo 2
  - $\Sigma = \{a, b, c\}$
  - Cadeias que contenham no máximo um b

# Expressões Regulares

- Exemplo 1
  - $\Sigma = \{a, b, c\}$
  - Cadeias que contenham exatamente um b
- Exemplo 2
  - $\Sigma = \{a, b, c\}$
  - Cadeias que contenham no máximo um b
- Exemplo 3
  - $\Sigma = \{a, b\}$
  - Cadeias compostas por um b cercado pela mesma quantidade de as $\rightarrow$ {b, aba, aabaa, ...}

# Expressões Regulares em Compiladores

- letra → A | ... | Z | a | ... | z
- digito → 0 | 1 | 2 | ... | 9
- id → letra ( letra | digito )*

- sinal → (+|-)
- numero → digito digito*
- inteiro → (sinal | ε) numero

# Expressões Regulares em Compiladores

- Notação: $r+ \equiv rr^*$

  $r? \equiv r \mid \varepsilon$

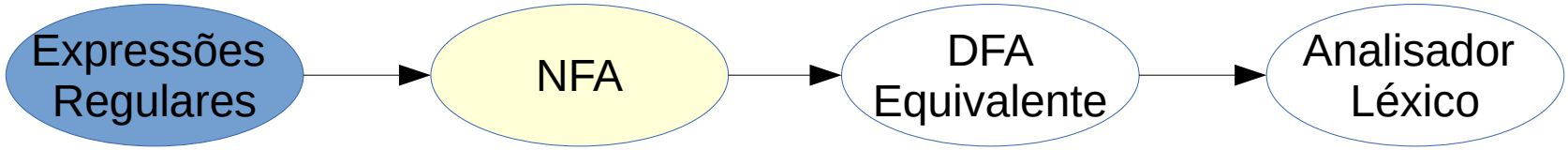
- Ex.: digitos $\rightarrow$ digito+
- inteiro $\rightarrow$ (sinal)? numero

# Exercício 4

- Escreva as expressões regulares que geram, em Cordel+1, os conjuntos:
  - De valores do tipo booleano (literais)
  - De valores numéricos (literais)
  - De identificadores

- Não se esqueça de seguir as regras da nossa linguagem!

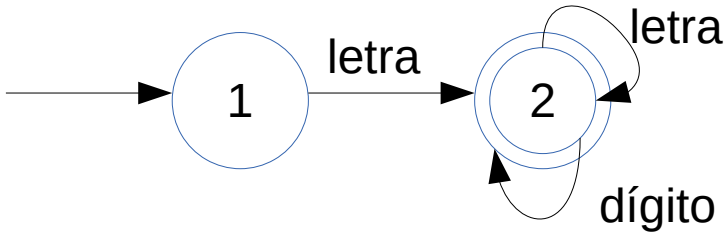
# Construindo um Analisador Léxico



# Autômatos Finitos

- Relação forte entre autômatos finitos e expressões regulares
- Usados para construir analisadores léxicos
- Descrevem o processo de reconhecimento de padrões

identificador = letra (letra | dígito)*



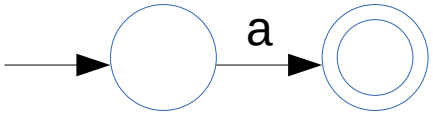
# De uma Expressão Regular para um NFA

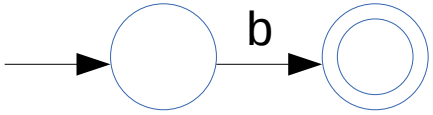
- Construção de Thompson
  - Usa ε-transições para unir os autômatos de cada pedaço de uma expressão regular
- Casos básicos:
  - Concatenação
  - Escolha entre alternativas
  - Repetição

# Exemplo 1

- Transformar uma expressão regular ab | a, segundo a construção de Thompson

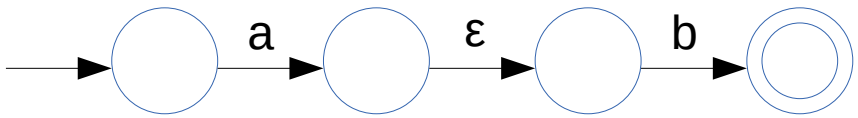
**a e b**



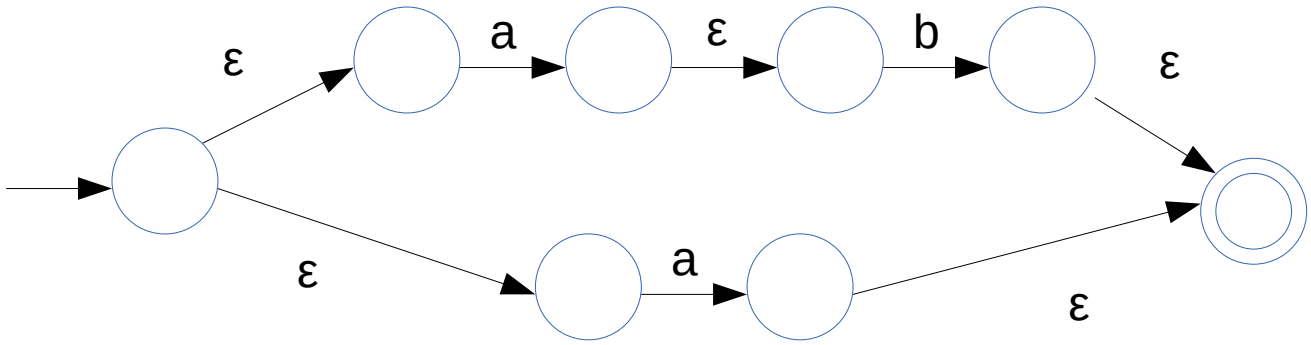



# Exemplo 1

**a b | a**




**a b | a**

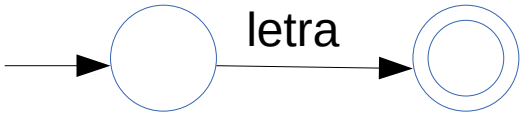


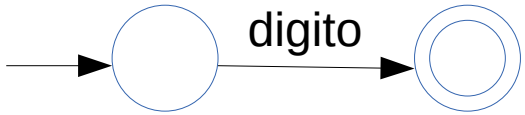
# Exemplo 2

- **letra (letra | digito)***

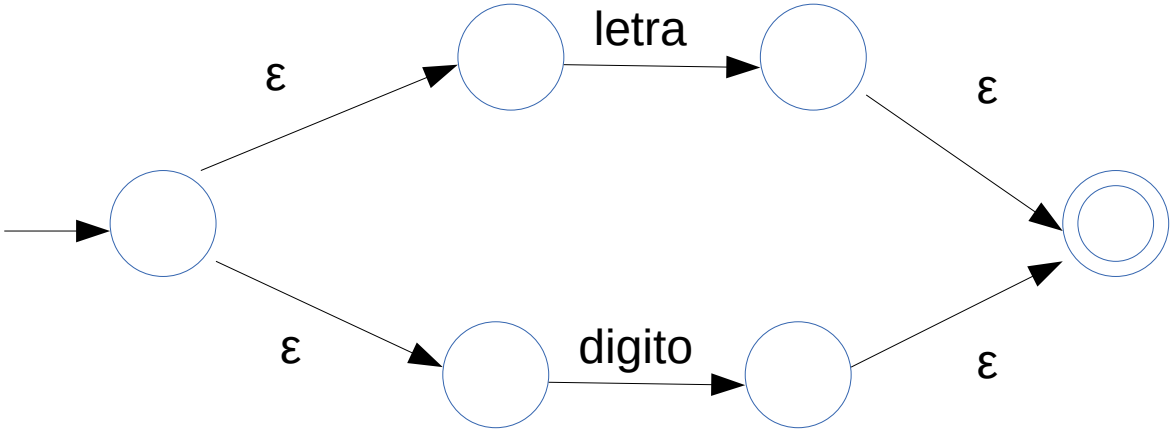
# Exemplo 2

- **letra (letra | digito)***



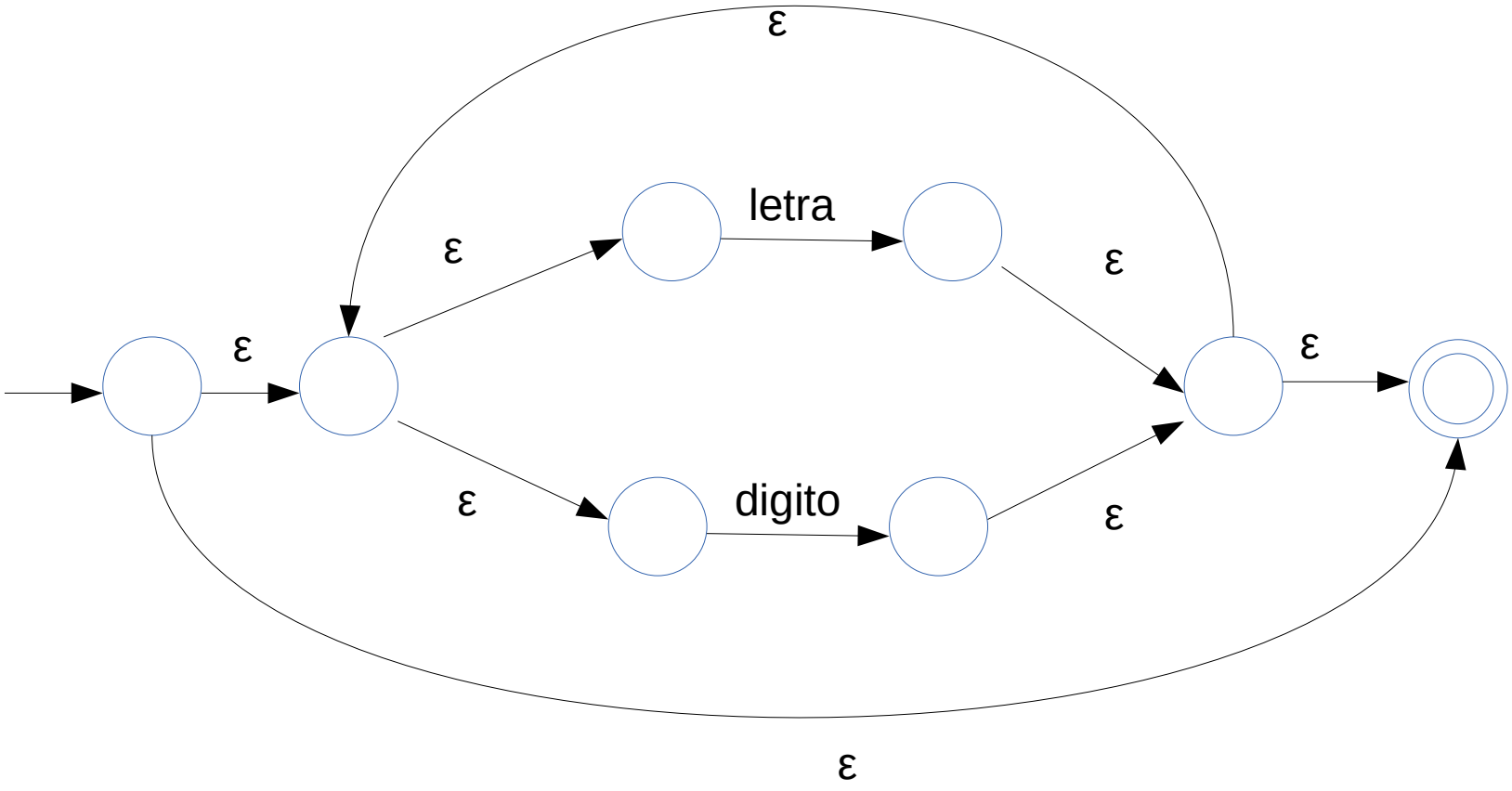




- **letra (letra | digito)***



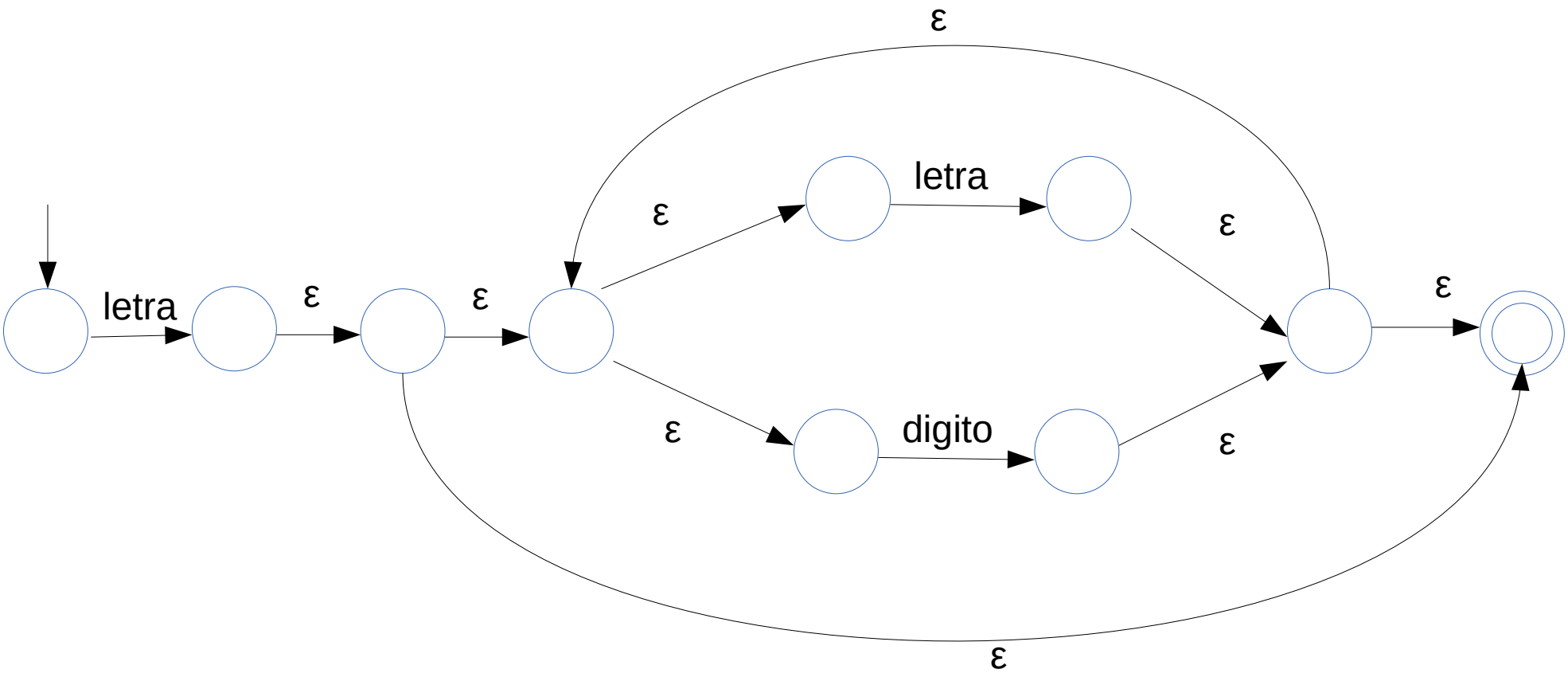
# Exemplo 2

- letra **(letra | digito)***



# Exemplo 2

- **letra (letra | digito)***



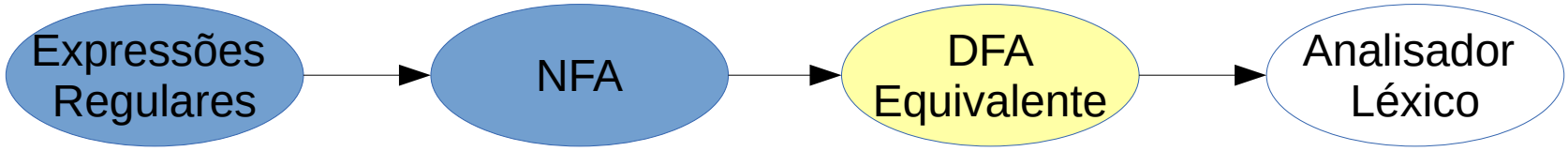
# Exercício 5 (HT)

- Usando a construção de Thompson, gere o NFA para a expressão regular que gera cadeias com números inteiros, abaixo:

inteiro = (**+**| **-** | ε) **digito digito***

Observação: assumam que o símbolo **digito** pertence ao alfabeto da linguagem gerada pela ER acima. Na prática, ele seria uma ER auxiliar, cuja linguagem é {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}.

# Construindo um Analisador Léxico



# De um NFA para um DFA

- Transformar o NFA em um DFA equivalente, ou seja, que aceite as mesmas cadeias
- Eliminar as ε-transições e as transições múltiplas
  - Conjunto de estados ao invés de estados únicos
  - Algoritmo é denominado **construção de subconjuntos**

# De um NFA para um DFA

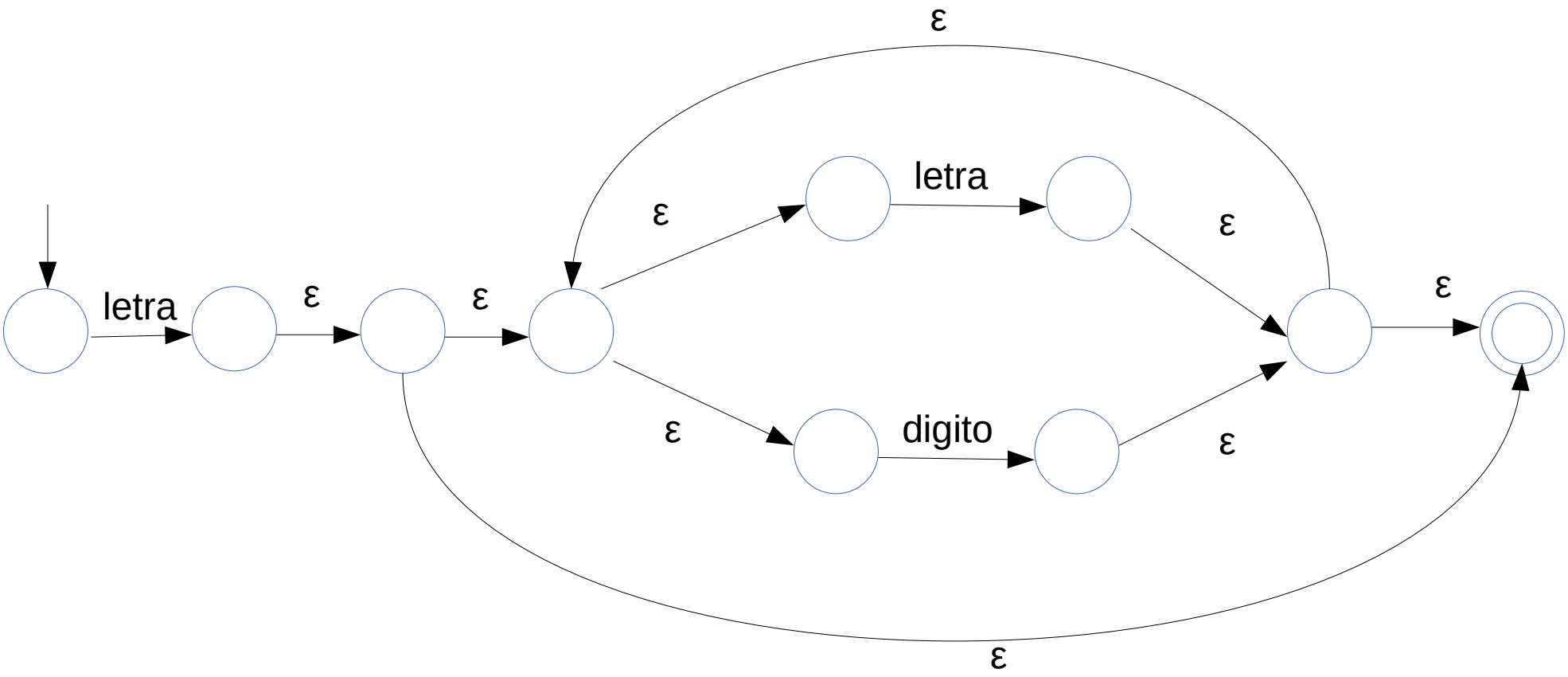
- Eliminar as ε-transições requer a construção de ε-fechos
- Definição de $\bar{s}$ (ε-fecho de um estado s)
  - $\bar{s}$ é o conjunto de estados alcançáveis a partir de s através de uma série de zero ou mais ε-transições

# De um NFA para um DFA

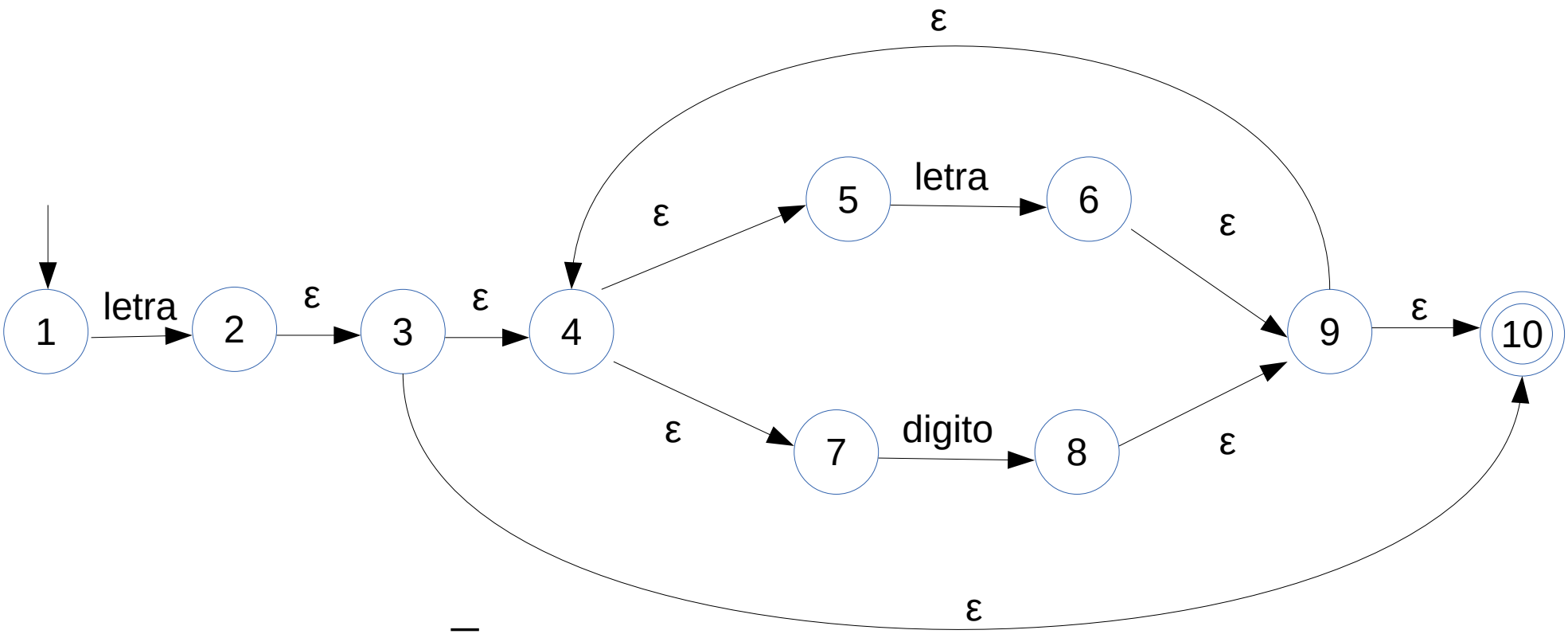
- Definição de $\overline{S}$ (ε-fecho de um conjunto de estados S)
  - $\overline{S}$ é a união dos ε-fechos de cada estado em S
- Construção do DFA $\overline{M}$ a partir de um NFA M

# Exemplo

- **letra (letra | digito)***

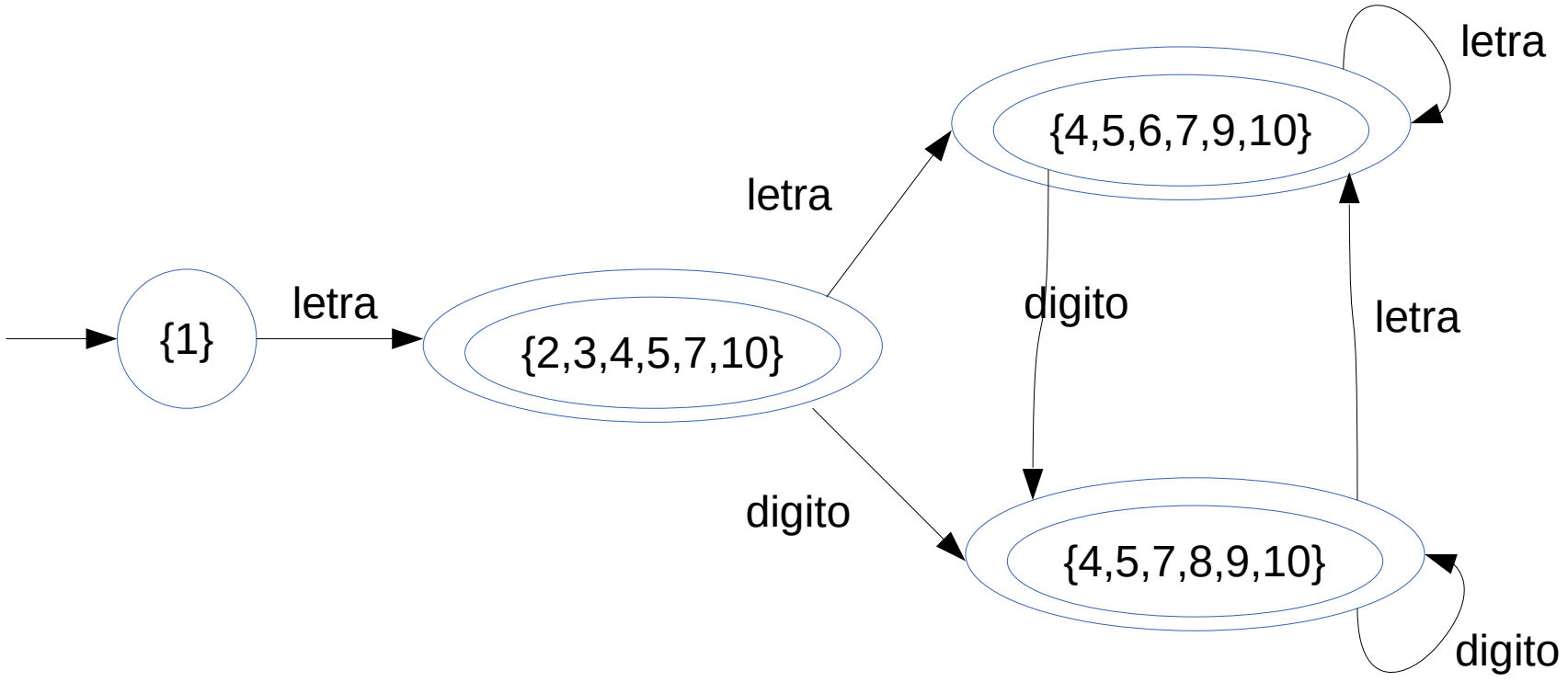


# Exemplo

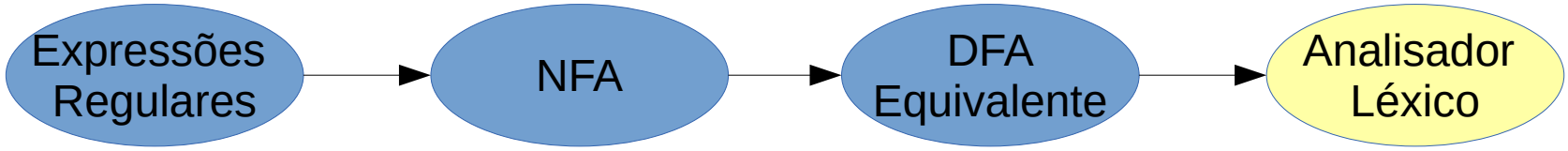



- O estado inicial é $\overline{1}$ = {1}
- Há uma transição em letra para $\overline{2}$ = {2, 3, 4, 5, 7, 10}
- A partir do estado 5, há uma transição em letra para $\overline{6}$ = {4, 5, 6, 7, 9, 10} e uma transição em dígito de 7 para $\overline{8}$ = {4, 5, 7, 8, 9, 10}

# Exemplo


{1}
letra
{2,3,4,5,7,10}
letra
digito
{4,5,6,7,9,10}
letra
digito
letra
{4,5,7,8,9,10}
digito

# Construindo um Analisador Léxico



# Implementação do Analisador Léxico

- Tabelas de transição podem ser usadas como estrutura de dados que representa o DFA
  - Código genérico para implementação do DFA dada a tabela
  - Código enxuto, porém necessidade de espaço de armazenamento aumenta com mais estados e caracteres
  - Mais lento (uso de matrizes esparsas)
- Uso de NFAs requer *backtracking*

# Exercício 6 (HT)

- Construa o DFA equivalente ao NFA gerado no Exercício 5.

# Referência

- Capítulo 2 do livro do Louden